

Chapter 9

Kubernetes in Action 1st Edition

Deployments

updating applications declaratively



신재근 (Jagger)

9장의 핵심 질문

서비스 중단 없이 버전을 올리고,
문제가 생기면 어떻게 빠르게 되돌릴까?

답

- Deployment로 선언적 롤링 업데이트
- rollout history / undo로 복구
- readiness + minReadySeconds로 불량 버전 차단

수동 업데이트의 한계

가능한 방식

- 삭제 후 재생성 (Recreate)
- Blue/Green (Service selector 전환)
- 수동 Rolling (구버전 축소 + 신버전 확대)

공통 문제

- 명령 수가 많고 순서 의존적
- 라벨/셀렉터 실수 위험
- 장애 시 즉시 복구 절차가 불명확

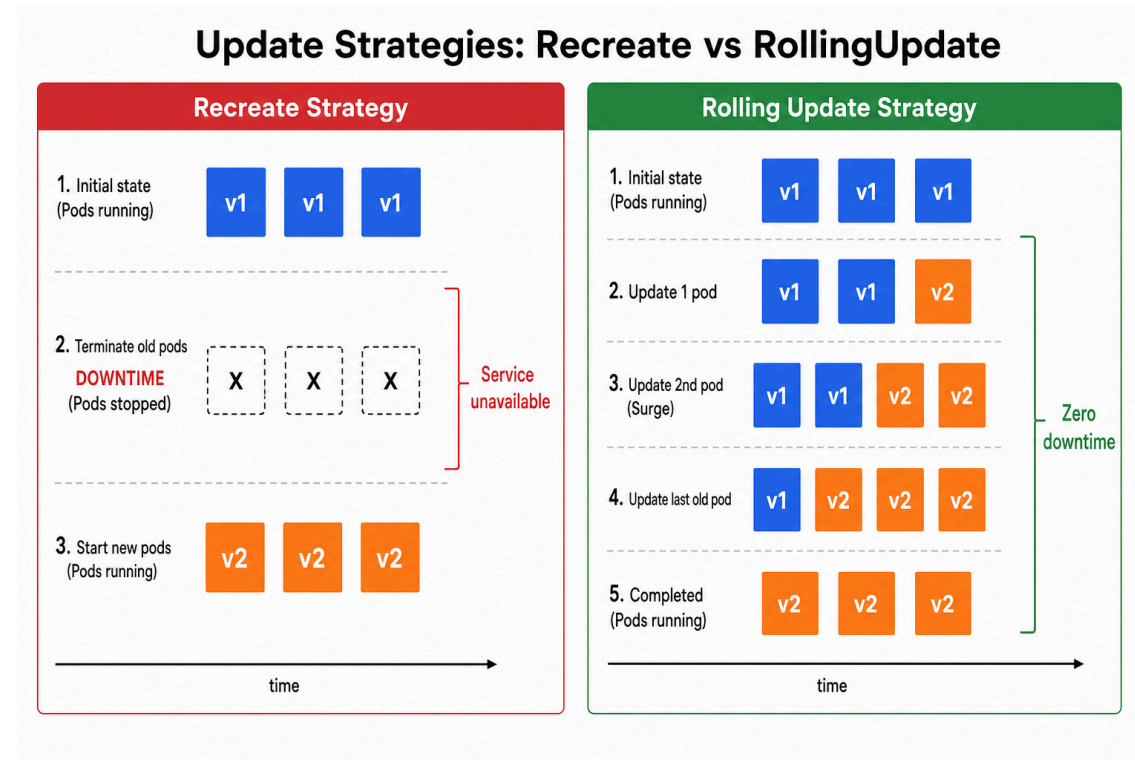
kubectl rolling-update 가 사라진 이유

옛날에는 `kubectl rolling-update old new --image=...` 명령이 있었음. 지금은 deprecated.

문제	설명
Imperative	명령어 시퀀스로 동작, desired state가 클러스터에 없음
클라이언트 측 동작	kubectl 죽으면 롤아웃 중단 (SSH 끊김 = 사고)
라벨 강제 변경	RC와 Pod 라벨에 <code>deployment</code> 필드 자동 추가 → 부작용
서버 재시작 시 복구 불가	진행 상태가 서버에 없음

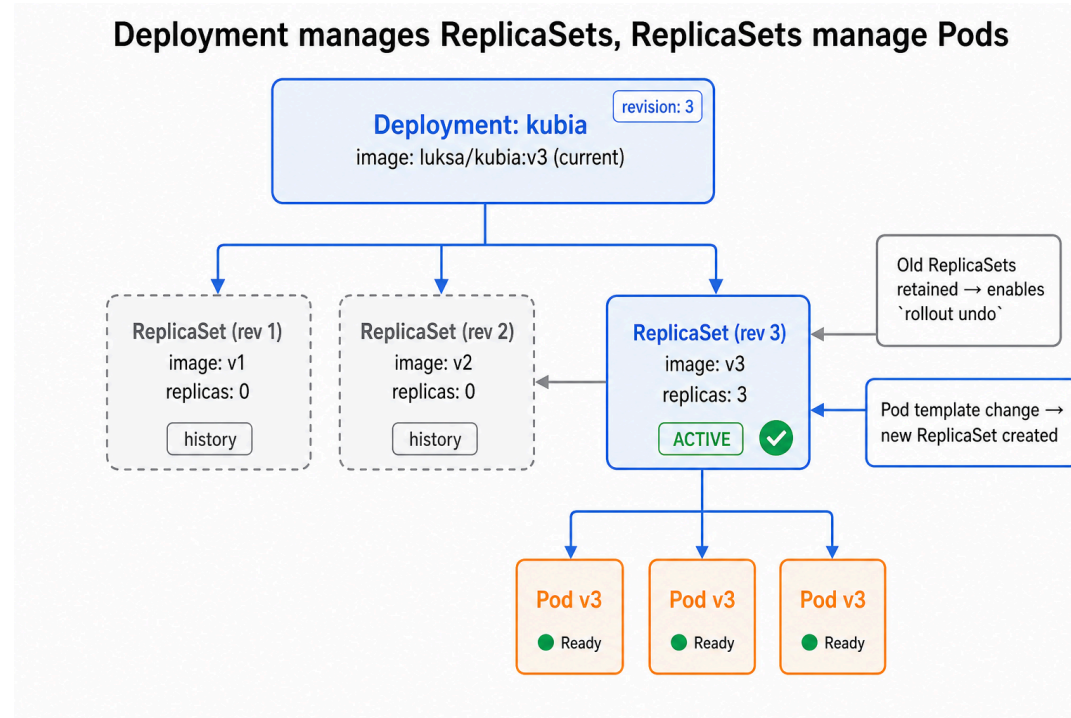
→ Deployment는 이 4가지를 모두 서버 측·선언적으로 해결

두 가지 업데이트 전략



- Recreate: 모든 구버전 종료 → 신버전 시작 (다운타임 발생)
- RollingUpdate: 점진 교체 → 항상 가용 Pod 유지

Deployment가 하는 일



- 사용자는 Deployment만 수정 → 내부적으로 새 ReplicaSet 생성·전환
- 이전 ReplicaSet은 `replicas: 0` 으로 보존 → rollback 가능

Pod template hash와 selector

```
$ kubectl get rs -l app=kubia
NAME                                DESIRED   READY   # v1 (rev 1, 보존)
kubia-68477c56d9                    0         0
kubia-7d64947c44                    3         3   # v2 (rev 2, 활성)
```

- RS 이름 끝부분 = Pod template의 해시값 (pod-template-hash)
- 같은 template이면 같은 hash → 새 RS 만들어지지 않음 (먹등)
- spec.selector 는 immutable — 한 번 정하면 변경 불가

롤아웃을 일으키는 6가지 방법

방법	명령 예시	용도
set image	<code>kubectl set image deploy/kubia nodejs=...:v2</code>	CI/CD 자동화
edit	<code>kubectl edit deploy/kubia</code>	임시 점검
patch	<code>kubectl patch deploy/kubia -p '...'</code>	스크립트
apply	<code>kubectl apply -f deploy.yaml</code>	GitOps 표준
replace	<code>kubectl replace -f deploy.yaml</code>	전체 교체
scale	<code>kubectl scale deploy/kubia --replicas=5</code>	트리거 아님

→ Pod template이 바뀌는 모든 변경은 자동 롤아웃 시작

Deployment strategy 명세

```
spec:
  strategy:
    type: RollingUpdate          # 또는 Recreate
    rollingUpdate:
      maxSurge: 1                # 또는 25%
      maxUnavailable: 0         # 또는 25%
    revisionHistoryLimit: 10
    progressDeadlineSeconds: 600
    minReadySeconds: 10
```

기본값: RollingUpdate , 25%/25% , historyLimit=10 , deadline=600s , minReady=0

실습: v1 배포

```
$ kubectl apply -f kubia-deployment-v1.yaml -f kubia-service.yaml
$ kubectl rollout status deployment/kubia
deployment "kubia" successfully rolled out
```

```
$ kubectl get pods -l app=kubia -o wide
```

NAME	READY	STATUS	IP	NODE
kubia-68477c56d9-5vhd6	1/1	Running	172.21.59.152	minikube-m04
kubia-68477c56d9-crjhm	1/1	Running	172.21.205.221	minikube-m02
kubia-68477c56d9-vmcfq	1/1	Running	172.21.151.30	minikube-m03

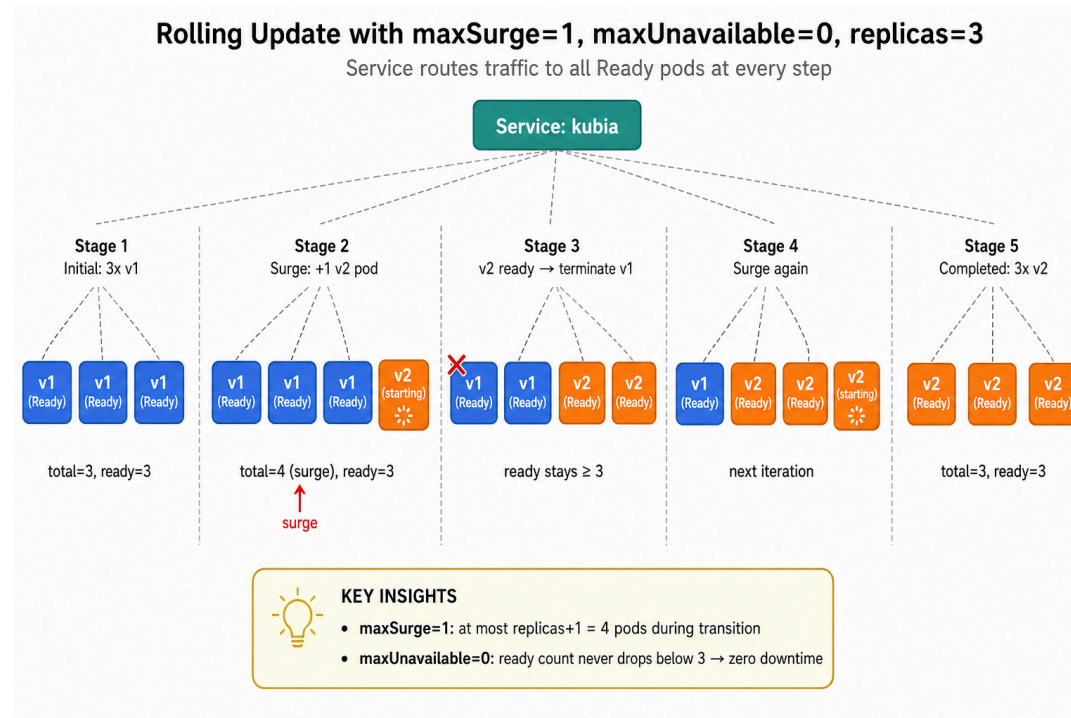
→ 3개 Pod이 3개 다른 노드에 분산 (Service가 라운드로빈)

실습: v1 트래픽 분포 (30회 호출)

```
12 This is v1 running in pod kuba-68477c56d9-5vhd6
9  This is v1 running in pod kuba-68477c56d9-vmcfq
9  This is v1 running in pod kuba-68477c56d9-crjhm
```

- 30회 = 12 + 9 + 9 → 거의 균등 분산 (라운드로빈 + 약간의 편차)
- baseline 확보: 5xx 0회, 모두 v1 응답

롤링 업데이트는 어떻게 진행되나



- **maxSurge=1** : 전환 중 최대 replicas+1 = 4개까지 허용
- **maxUnavailable=0** : Ready 개수가 3 이하로 떨어지지 않음 → 무중단

실습: v2 롤링 업데이트

```
$ kubectl set image deployment/kubia nodejs=luksa/kubia:v2  
deployment.apps/kubia image updated
```

t = 2초 (진행 중):

replicaset.apps/kubia-68477c56d9	1/1/1	# v1 (구버전)
replicaset.apps/kubia-7d64947c44	3/2/2	# v2 surge, 1개 Pending
pod/kubia-68477c56d9-5vhd6	1/1	Terminating
pod/kubia-7d64947c44-8rvhx	0/1	Pending ← surge 중인 Pod

t = 8초 (완료):

kubia-68477c56d9	0/0/0
kubia-7d64947c44	3/3/3

events로 본 maxSurge=1, maxUnavailable=0 동작

```
$ kubectl get events --field-selector involvedObject.name=kubia
```

LAST SEEN	REASON	MESSAGE
32s	ScalingReplicaSet	Scaled up kubia-7d64947c44 from 0 to 1
31s	ScalingReplicaSet	Scaled down kubia-68477c56d9 from 3 to 2
31s	ScalingReplicaSet	Scaled up kubia-7d64947c44 from 1 to 2
30s	ScalingReplicaSet	Scaled down kubia-68477c56d9 from 2 to 1
30s	ScalingReplicaSet	Scaled up kubia-7d64947c44 from 2 to 3
29s	ScalingReplicaSet	Scaled down kubia-68477c56d9 from 1 to 0

→ **+1 → -1 → +1 → -1 → +1 → -1** 패턴 정확히 기록

과도 구간에서 캡처된 혼합 트래픽

resume 직후 30회 호출 (v2→v4 전환 중):

```
9 This is v4 running in pod kubaia-5bdf99478b-m6k6k
8 This is v4 running in pod kubaia-5bdf99478b-wnlj9
7 This is v4 running in pod kubaia-5bdf99478b-hf8bs
6 This is v2 running in pod kubaia-7d64947c44-blrl7
```

→ v4 24회 + v2 6회 = 80:20 혼합 응답 실제 캡처

해석:

- 무중단 = 가용성 유지 + 짧은 버전 공존
- v2 종료 직전 마지막 1개 Pod이 6회 받은 모습

실습: 불량 v3 배포 (probe 없이)

```
$ kubectl set image deployment/kubia nodejs=luksa/kubia:v3
$ kubectl rollout status deployment/kubia
deployment "kubia" successfully rolled out
```

← "성공" 처리됨!

60회 호출 결과 (status code 카운트):

```
36 500
24 200
```

→ 사용자 요청의 60% 가 5xx 노출 ⚠

실습: 즉시 롤백 + revision 단조 증가 확인

```
$ kubectl rollout undo deployment/kubia
deployment.apps/kubia rolled back

$ kubectl get deployment kubia -o jsonpath='{.spec.template.spec.containers[0].image}'
luksa/kubia:v2
```

롤백 전 history: 롤백 후 history:

1	initial v1	1	initial v1
2	upgrade to v2	3	upgrade to v3 (broken)
3	upgrade to v3 (broken)	4	rollback to v2 ← NEW

→ rev 2가 사라지고 rev 4 새로 등장: 같은 image(v2)지만 새 revision

revision 관리: history / change-cause / undo

```
$ kubectl rollout history deployment/kubia
REVISION  CHANGE-CAUSE
1         initial v1
2         upgrade to v2
3         upgrade to v3 (broken)
4         rollback to v2
```

- `revisionHistoryLimit: 10` (기본) — 보존되는 RS 개수
- CHANGE-CAUSE 채우기: `kubernetes.io/change-cause` annotation 권장 (`--record` 는 deprecated)
- `kubectl rollout undo deploy/kubia --to-revision=1` — 특정 revision으로 점프

CHANGE-CAUSE annotation 예시

```
metadata:
  name: kubia
  annotations:
    kubernetes.io/change-cause: "upgrade to v3 with readinessProbe"
spec:
  ...
```

- YAML에 박아두면 **kubectl apply** 만으로 history에 기록됨
- Git PR 메시지와 일치시키면 추적 일관성 향상

maxSurge / maxUnavailable 시나리오 비교

replicas=10 기준:

설정	최대 Pod	최소 Ready	용도
surge=1, unavailable=0	11	10	무중단 우선 (SLA 중요)
surge=0, unavailable=1	10	9	리소스 절약
surge=25%, unavailable=25%	13	8	기본값 (균형)
surge=100%, unavailable=0	20	10	최고 속도 (Blue-Green 유사)

규칙:

- 둘 다 0은 불가능 (롤아웃 진행 못함)
- 백분율: surge는 올림, unavailable은 내림

실습: Pause → set image → Conditions

```
$ kubectl rollout pause deployment/kubia
$ kubectl set image deployment/kubia nodejs=luksa/kubia:v4

$ kubectl describe deployment kubia | grep -A 4 Conditions
Conditions:
  Type            Status    Reason
  Available        True      MinimumReplicasAvailable
  Progressing      Unknown   DeploymentPaused
NewReplicaSet:    <none>
```

← 핵심
← 새 RS 생성도 보류

→ pause 먼저 하면 NewReplicaSet 생성도 보류됨 (트래픽 100% v2 유지)

Pause 중 트래픽 (60회 호출)

```
23 This is v2 running in pod kuba-7d64947c44-blrl7
19 This is v2 running in pod kuba-7d64947c44-8jwkf
18 This is v2 running in pod kuba-7d64947c44-rp4v9
```

→ 60회 모두 v2 (pause→set image 순서 → 새 RS가 안 만들어졌기 때문)

resume:

```
$ kubectl rollout resume deployment/kuba
$ kubectl rollout status deployment/kuba
deployment "kuba" successfully rolled out
```

Pause/Resume vs 정식 카나리

항목	<code>rollout pause</code>	정식 카나리
트래픽 비율 제어	maxSurge로 결정 (정밀 X)	5% / 10% 정밀 가중치
두 버전 격리	같은 Service / 같은 라벨	별도 Deployment + Service
롤백	<code>rollout undo</code>	카나리 Deployment 삭제
도구	kubectl 만으로 가능	Argo Rollouts / Flagger / Istio

→ pause는 임시 점검용, 정식 카나리는 별도 도구

4중 품질 게이트

필드	정확한 의미	권장 값
<code>readinessProbe</code>	실패 시 endpoint 제외 (트래픽 차단)	항상 설정
<code>minReadySeconds</code>	Ready를 N초 유지해야 Available로 인정	5~30초
<code>maxUnavailable: 0</code>	구버전 종료 금지 → 롤아웃 STUCK	SLA 중요 시
<code>progressDeadlineSeconds</code>	진행 없으면 ProgressDeadlineExceeded	600초(기본)

→ 넷이 함께 동작해야 결함 버전이 자동 차단됨

Available vs Progressing Condition

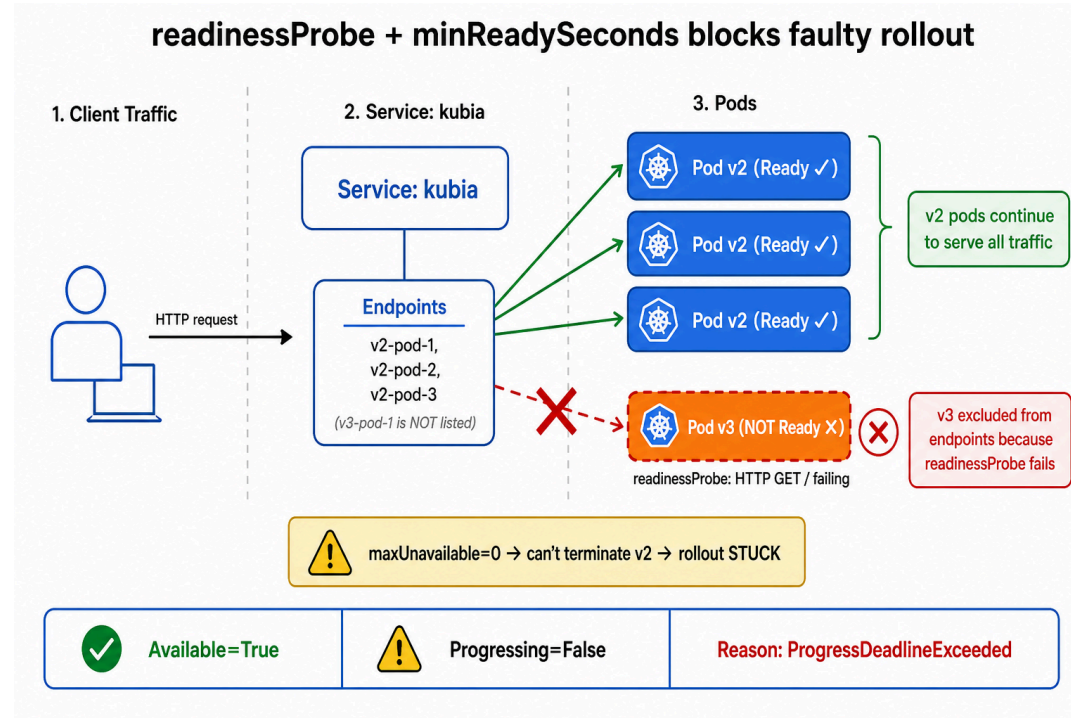
```
$ kubectl describe deploy/kubia | grep -A5 Conditions
Conditions:
```

Type	Status	Reason
Available	True	MinimumReplicasAvailable
Progressing	False	ProgressDeadlineExceeded

Condition	의미
Available=True	사용자 트래픽 처리 중 (안전)
Progressing=True	롤아웃 정상 진행
Progressing=False, ProgressDeadlineExceeded	운영자 개입 필요

→ 핵심: 불량 롤아웃이 멈춰도 **Available=True** 유지

차단 메커니즘 한눈에



- 신버전 Pod는 readiness 실패 → endpoint 제외 → 사용자 트래픽 차단
- `maxUnavailable=0` → 구버전을 못 내림 → 롤아웃 STUCK

실습: 불량 롤아웃 자동 차단

```
$ kubectl apply -f kubia-deployment-v3-readiness.yaml
$ kubectl rollout status deployment/kubia --timeout=75s
Waiting for deployment "kubia" rollout to finish: 1 old replicas are pending termination...
...
error: deployment "kubia" exceeded its progress deadline ← 정확히 60초 후
```

exit code: 1 → CI/CD 파이프라인 실패 처리 가능

```
$ kubectl describe pod kubia-5884666799-gck29 | tail -3
Events:
  Warning   Unhealthy   59s (x25 over 82s)   kubelet
    Readiness probe failed: HTTP probe failed with statuscode: 500
```

→ 82초 동안 25번 probe 실패 (periodSeconds: 1)

차단 중 서비스 응답 (60회 호출)

```
60 200 This is v2 running in pod kuba-7d64947c44-cgm5m
```

→ 60/60 = 100% v2 정상 응답, 5xx 0회

Endpoints 직접 확인:

```
$ kubectl get endpoints kuba
NAME      ENDPOINTS                AGE
kuba      172.21.151.37:8080       16m
```

← v2 Pod IP만 등록

→ v3 Pod이 살아있어도 endpoint에 없으니 트래픽 단 1회도 v3로 가지 않음

같은 v3 결함, 다른 결과 — 정량 비교

같은 `luksa/kubia:v3` 결함 버전, 60회 호출 결과:

항목	probe 없음 (Step 3)	probe + minReady=10 + unavailable=0 (Step 5)
롤아웃 결과	"성공" 처리	<code>ProgressDeadlineExceeded</code> 자동 정지
5xx 응답 횟수	36 / 60 = 60% ⚠	0 / 60 = 0% ✓
v3 Pod의 운명	endpoint 등록 → 트래픽 받음	endpoint 미등록 → 트래픽 차단
감지 방법	사후 대시보드 (지연)	자동 (60초 후 exit 1)
CI/CD 연결	불가 ("성공" 처리됨)	가능 (비-0 종료 코드)
운영자 부담	알람 → 분석 → 롤백	알람 → 즉시 undo

→ 같은 결함 → 사용자 영향 60% vs 0% = 게이트 의무화의 정량 근거

불량 롤아웃 중단(Abort)

```
kubectl rollout undo deployment/kubia  
kubectl rollout status deployment/kubia  
kubectl get deployment kubia -o jsonpath='{.spec.template.spec.containers[0].image}'
```

```
luksa/kubia:v2
```

무중단의 전제: 버전 공존 호환성

롤링 업데이트 중에는 v1과 v2가 반드시 동시에 동작한다.

영역	보장해야 할 것
DB 스키마	v1·v2 모두 읽기/쓰기 가능 (expand-contract 패턴)
API 계약	새 필드는 optional, 기존 필드는 유지
메시지 큐	v2가 v1 포맷을 deserialize 가능
캐시 키	버전 충돌하지 않게 prefix 분리

→ 버전 N과 N-1이 항상 호환되도록 설계해야 무중단이 의미를 가짐

운영 체크리스트

- Deployment 단일 진입점 — RC/RS 직접 사용 금지
- image tag immutable (`v1` , `v2` 또는 commit SHA)
- `readinessProbe` 의무화 — 가장 중요한 단일 안전장치
- `minReadySeconds` 5~30초 (워밍업 + 안전 마진)
- `maxUnavailable: 0` 을 SLA 중요 서비스 기본값으로
- `revisionHistoryLimit` , `progressDeadlineSeconds` 명시 (기본값 의존 X)
- CHANGE-CAUSE annotation을 CI 파이프라인으로 자동 채움
- `progressDeadlineSeconds` 알람 → 자동 알림 연결
- 장애 감지 → `kubectl rollout undo` 를 runbook 1번 항목
- DB 스키마는 expand → contract (롤링 중 호환 보장)

핵심 요약

Deployment는

무중단 전환(rolling) + 속도/가용성 제어 + 즉시 롤백
을 하나의 리소스로 제공한다.

Q&A

질문	답변
RS 직접 운영하면 안 되나?	가능하지만 업데이트/복구 운영비용이 큼
rollout undo는 언제?	사용자 영향 장애 신호 감지 즉시
probe만 있으면 충분?	아니오. minReadySeconds와 함께 써야 안정
pause가 카나리 정식 해법인가?	임시 점검에는 유용, 정식 카나리는 보통 별도 Deployment 2개